

```
$ scout ask "best stand for tomorrow?"
```

```
RECOMMENDATION: Bean Field stand  
WINDOW: 06:00 – 07:00 (civil dawn)  
CONFIDENCE: 0.94
```

```
$ scout fleet sync --firmware
```

```
8 cameras: firmware 2.2 (UTC timestamps)  
4 cameras: firmware 2.1 (Central timestamps)  
STATUS: all cameras reporting. no schema errors.
```

```
| $ scout ask "best stand for tomorrow?"
```

```
RECOMMENDATION: Bean Field stand  
WINDOW: 11:00 – 12:00  
CONFIDENCE: 0.94 ■
```

```
ERRORS: 0          WARNINGS: 0          RUNTIME: 1.2s
```


AgentOps for Data Teams

*Observability, Evals, and Control
for Production AI*

BRIAN HAYDIN

SOLUTION ARCHITECT • CONCURRENCY

brianhaydin.com



Data-Driven Wisconsin 2026
• MSOE Milwaukee
• Session 3, 3:00 PM
• Track 2

Same Tree.
Same Angle.
Same Time of Day.



SAME TREE. SAME ANGLE. SAME TIME OF DAY.

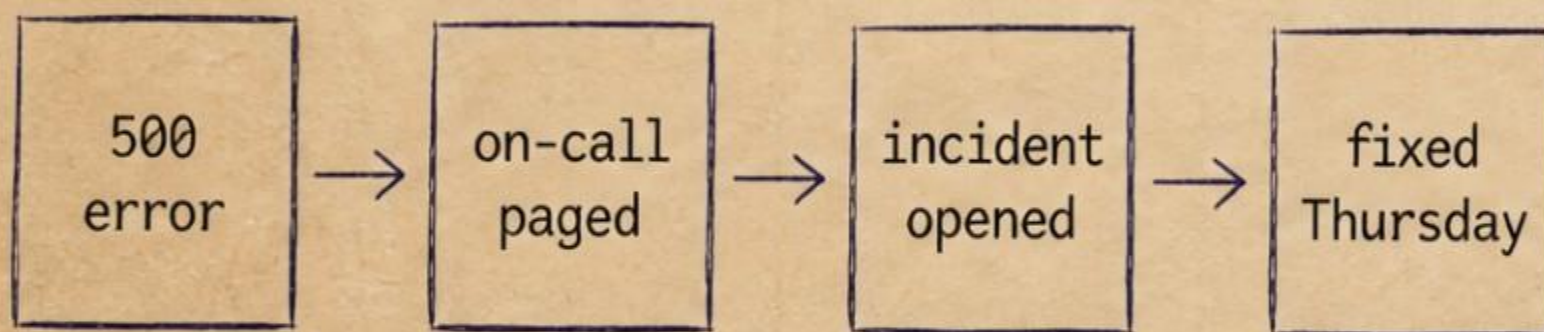
TRAIL CAMERA	AGENTOPS
Fixed position, fixed angle	<i>Golden prompts</i>
The whole card, not the trophy photo	<i>Tracing</i>
Last year's card vs. this year's	<i>Regression testing</i>
Confirming before you release	<i>Approval gate</i>
Climbing down when the wind switches	<i>Kill switch</i>

Hope is not a scouting strategy. Prompt-and-pray isn't either.

Production AI Doesn't Fail Loudly

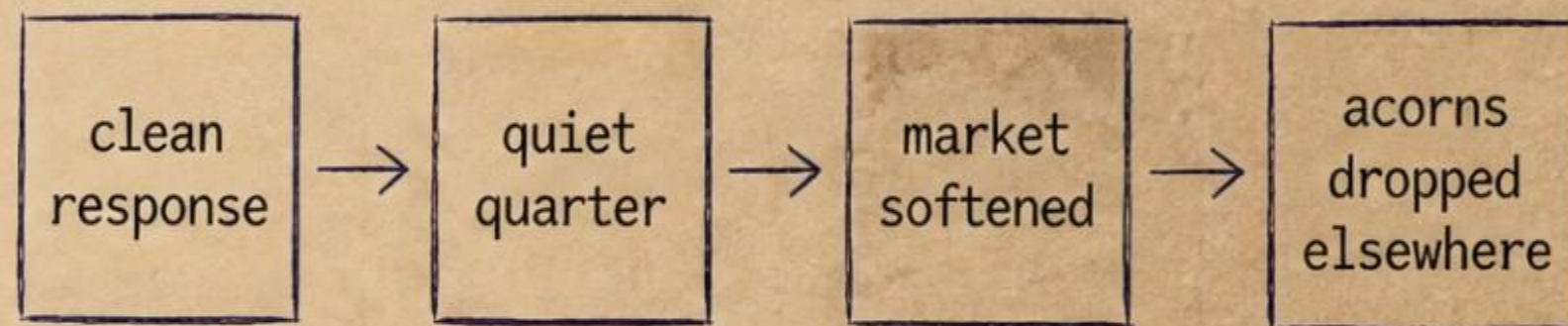
the antagonist
is plausibility

LOUD FAILURE



→ generates a TICKET

PLAUSIBLE FAILURE



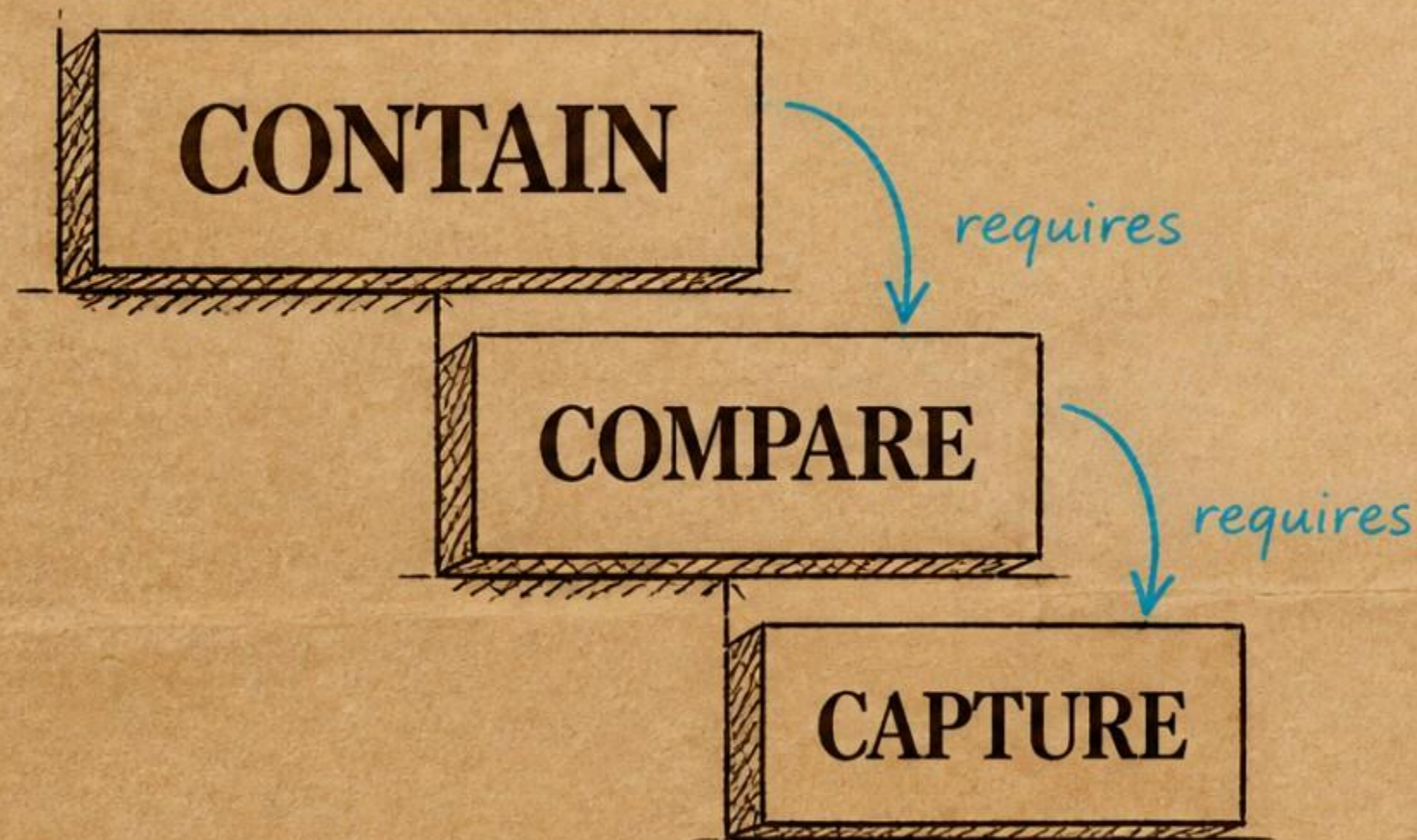
→ generates an EXPLANATION

*Production AI doesn't fail loudly — it fails plausibly,
and plausible failures survive because nobody files a ticket.
AgentOps is how you generate the ticket.*

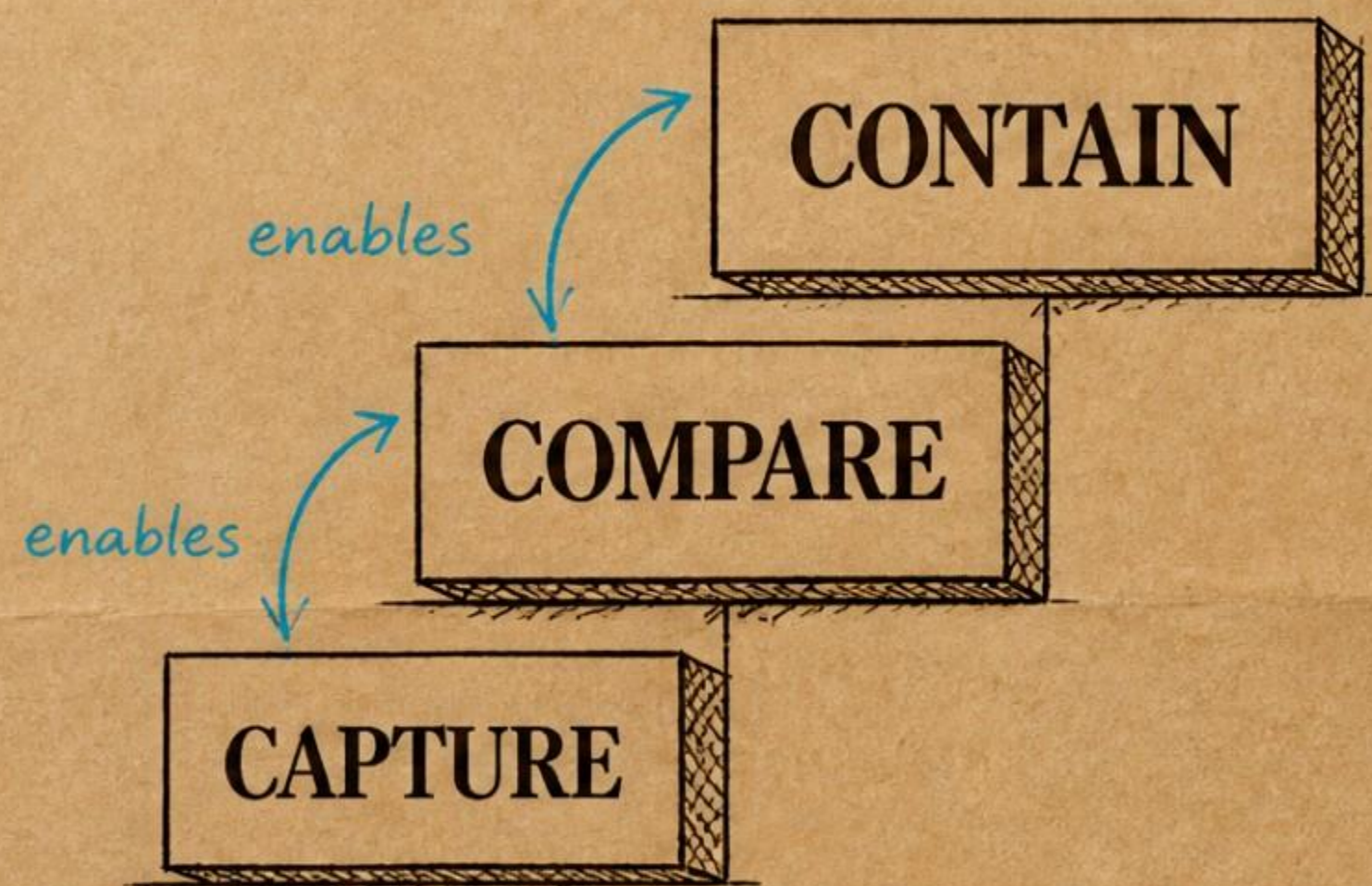
What You Want Is the Reverse of What You Build

*start at the
bottom of the
staircase*

WHAT YOU WANT →



→ WHAT YOU BUILD



A kill switch you never trip is theater.

Three Things I'm Not Claiming

1. ~~This is a new discipline.~~
Most of it is borrowed from data engineering.

2. ~~You need new tooling.~~
Something you own already runs an eval suite.

3. ~~Twenty test cases is coverage.~~
But most teams have zero, and zero is the problem.

MLflow · Agent Evaluation · Arize · LangSmith · Braintrust · Foundry evaluations

vendor-unfocused, on purpose

The Whole Card, Not the Trophy Photo

SPAN ANATOMY · A GREEN TURN

```
$ scout trace show trace_id=1c9d...a02
```

```
└─ turn.request1120ms status=OK
  │ └─ planner.decide284ms
  │   │ intent: "peak_activity_window"
  │   │ reasoning: "aggregate by hour of day"
  │   └─ tool.retrieve_detections762ms
  │     │ sql: SELECT DATE_TRUNC('hour',
  │     │ captured_at) h, COUNT(*) c
  │     │ FROM detections
  │     │ WHERE season='2025' GROUP BY 1;
  │     │ rows_returned: 47
  │   └─ response.compose74ms
  └─ turn.complete1120ms
```

```
ERRORS: 0 WARNINGS: 0 RUNTIME: 1120ms
```

the planner span

what the model decided to do, and why.

the tool span

which tool. duration. args.

the SQL, verbatim

the exact statement,
the exact parameters,
the exact rows returned.

The failure mode isn't the exception. It's the agent doing everything correctly against data that moved.

A Trace Is Just Another Fact Table

no new tool.
no new dialect.

OTel span schema

trace_id	string
span_id	string
parent_id	string
name	string
kind	enum
duration_ms	int
status	enum
attributes	json

same rows.
familiar joins.

SQL you already write

```
SELECT tool_name,  
       p95(duration_ms) AS p95_ms,  
       COUNT(*) AS calls  
FROM   agent_traces  
WHERE  ts >= NOW() - INTERVAL '7 days'  
GROUP BY tool_name  
ORDER BY calls DESC;
```

retrieve_detections	842ms	1,247
query_forecast	314ms	683
text_writer_gated	12ms	104

OpenTelemetry → your lakehouse → the SQL you already write.

Not All Eval's Assert on the Answer

<u>EVAL TYPE</u>	<u>ASSERTS ON</u>	<u>NEEDS THE TRACE?</u>
Outcome	The final answer	No
Process	Which tool, which retrieval, how many steps	THE TRACE IS THE DATASET
Efficiency	Tokens, cost per resolved outcome, latency	THE TRACE IS THE DATASET
Data-layer	Distributional invariants in one source	No
Cross-signal	Consistency between independent sources	Sometimes
LLM-as-judge	Tone, completeness, did it answer the question	Sometimes

You cannot eval what you did not capture.

2 REQUIRE THE
TRACE + 2
CAN USE IT

When the Feed Isn't Yours

SECOND DOMAIN. SAME FAILURE.

MARINE FORECAST

1 ft

wave height

issued 14:00 CT



CANNOT BOTH BE TRUE

WIND OBSERVATION

20 kt

east, gusting

observed 14:12 CT

WEST
SHORE



60 miles of open fetch



Different agent. Different domain. Same plausible failure.

*assert IMPOSSIBILITY,
not disagreement*

Accuracy Is the Wrong Number

ESCALATION RATE

6.2%



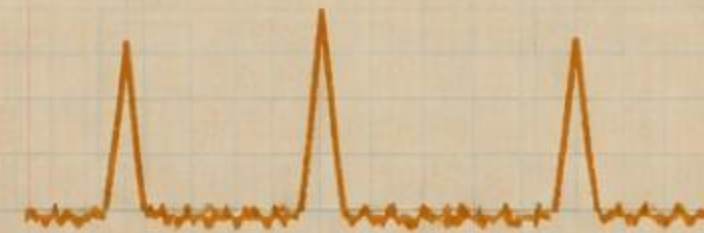
UNRESOLVED-TURN RATE

2.1%



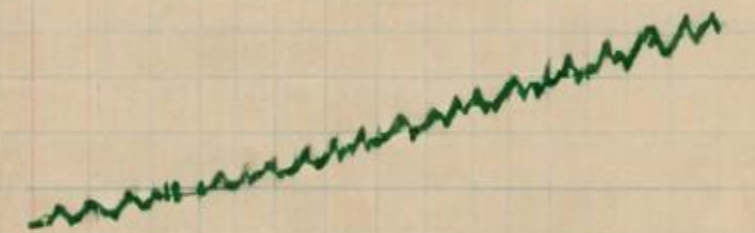
TOOL-ERROR RATE

0.9%



HELD-WRITE RATE

4.4%



Accuracy is a number you compute after you already know the answer.

all four → trace-derived • leading, not lagging

Sam McCune callback —
9:45, same room

What's the Contract When the Output Isn't Schema-Checkable?

DETERMINISTIC PIPELINE

- ☒ schema
- ☒ types
- ☒ nullability
- ☒ freshness



→ contract fires at ingestion

NON-DETERMINISTIC TURN



→ every row loaded clean

Evals are the contract for non-deterministic systems.

same job — catch it before it reaches a human — different mechanism

You Already Have the Twenty Questions.

capture-before-compare,
part two

PRODUCTION LOGS

[2026-08-04 14:32] user_query: "what stand tomorrow?"

the questions people actually ask

INCIDENT TICKETS

TICKET #4482 — "agent gave me a weird answer"

already prioritized by a human's complaint

USER WORKFLOWS

PLAN A HUNT → compare options → recommend a stand

EXPLAIN WHY → show evidence → surface uncertainty

PREPARE ACTION → assemble the plan → hand off for approval

the jobs the system has to complete

SME RULES

- * never recommend the wrong wind
- * never send text without approval
- * never suggest closed-season game

the ones asserted exactly

Not from imagination. From four sources you already have.

an afternoon, not a quarter

Assert on Numbers. Never on Prose.



X don't assert on prose

```
case: peak_activity_window
expect:
  answer_contains: "peak activity is around dawn"
```

one prompt improvement breaks this



✓ assert on numbers

```
case: peak_activity_window
expect:
  detections: 43
  verdict: NO
  peak_hour_window: [5, 8]
```

an integer, an enum, a range

REGENERATE WHEN

- business rule genuinely changed
- data model changed by design
- added capability
- case is obsolete

NEVER REGENERATE BECAUSE

the suite went red and you want it green

→ that's not a golden set. that's a screenshot of the bug.


```
$ scout eval run --suite golden --strict
```

```
> loading 20 cases from golden/
```

```
> 8 factual · 5 temporal · 3 rule · 2 refusal · 2 write-gate
```

```
> executing against production warehouse...
```

FACTUAL	[■■■■■■■■■■]	8/8	PASS
TEMPORAL	[□□□□□□□□]	0/5	FAIL
RULE	[■■■■■■■■■■]	3/3	PASS
REFUSAL	[■■■■■■■■■■]	2/2	PASS
WRITE-GATE	[■■■■■■■■■■]	2/2	PASS

TOTAL		15/20	

```
➤ GATE: BLOCKED. This build does not ship.
```

```
ERRORS: 0    WARNINGS: 0    RUNTIME: 6.4s    HELD BY POLICY: 0 ■
```


That's Not an Alarm. That's a Diagnostic.

2-day investigation.
→ 20 minutes

FACTUAL		8/8
TEMPORAL		0/5
RULE		3/3
REFUSAL		2/2
WRITE-GATE		2/2

every failure —
one category.
that's a
diagnostic,
not an alarm.

- 2 of 5 failed on VOLUME ONLY — peak hour didn't move
- 65% of morning detections vanished (Nov DST + drifted timestamps)

Assert the shape, not just the answer.


```
$ scout trace show trace_id=8a4f...eb2 ---attrs
```

```
└─ turn.request 1240ms
  └─ planner.decide 312ms
    intent: "peak_activity_window"
  └─ tool.retrieve_detections 842ms
    sql: SELECT DATE_TRUNC('hour', captured_at) h,
           COUNT(*) c FROM detections
           WHERE season='2026' GROUP BY 1;
    rows_returned: 43
    filters: season=2026, camera_id IN (...)
    ---- attributes-----
      camera.firmware_versions: [2.1, 2.2, 2.2] ◀
      camera.timezone_mixed: true
  └─ response.compose 86ms
└─ turn.complete 1240ms status=OK
```

```
ERRORS: 0      WARNINGS: 0      RUNTIME: 1240ms
```

```
=====
DATA-LAYER INVARIANT – crepuscular activity check
```

```
> assert: >=80% of detections within ±100 min of civil dawn/dusk
```

```
season 2023: 92% ✓
```

```
season 2024: 91% ✓
```

```
season 2025: 93% ✓
```

```
season 2026: 33% ✗ <-- FAIL
```

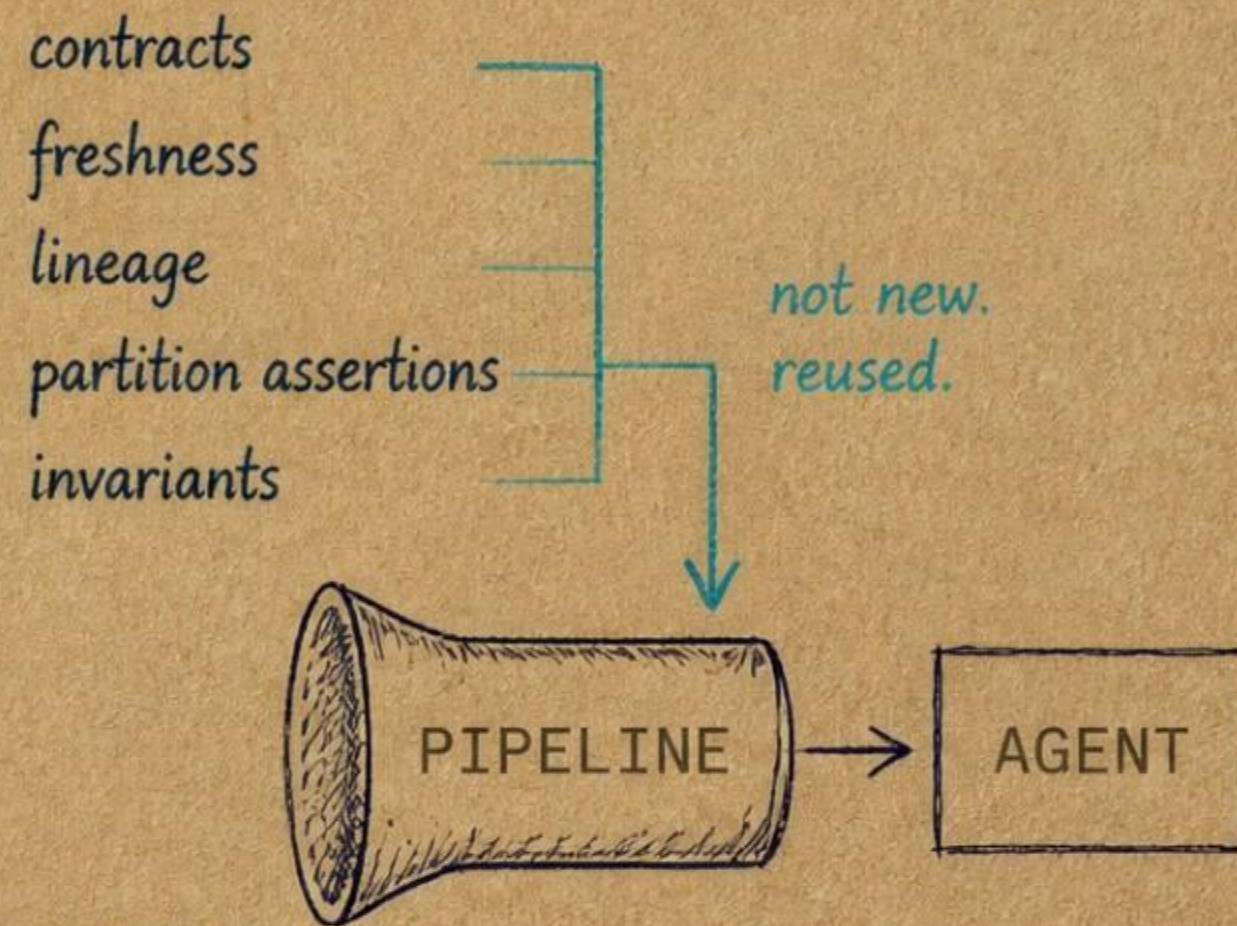
```
> grouped by camera_id + firmware_version:
```

```
cameras below threshold: 8 of 12
```

```
ALL 8 on firmware 2.2 ■
```


Most agent failures are data failures wearing a costume.

Your agent is a downstream consumer. Instrument it like one.



somebody had to know what normal looks like — in advance. that's the work.

Gates Everywhere Means Gates Nowhere

compliance
artifact
that fools
nobody



GATE IT

- ☒ irreversible
- ☒ reaches a human
- ☒ moves money
- ☒ changes state you can't roll back

→ human approval required

DON'T

- ~~reads~~
- ~~queries~~
- ~~internal computations~~

→ agent proceeds

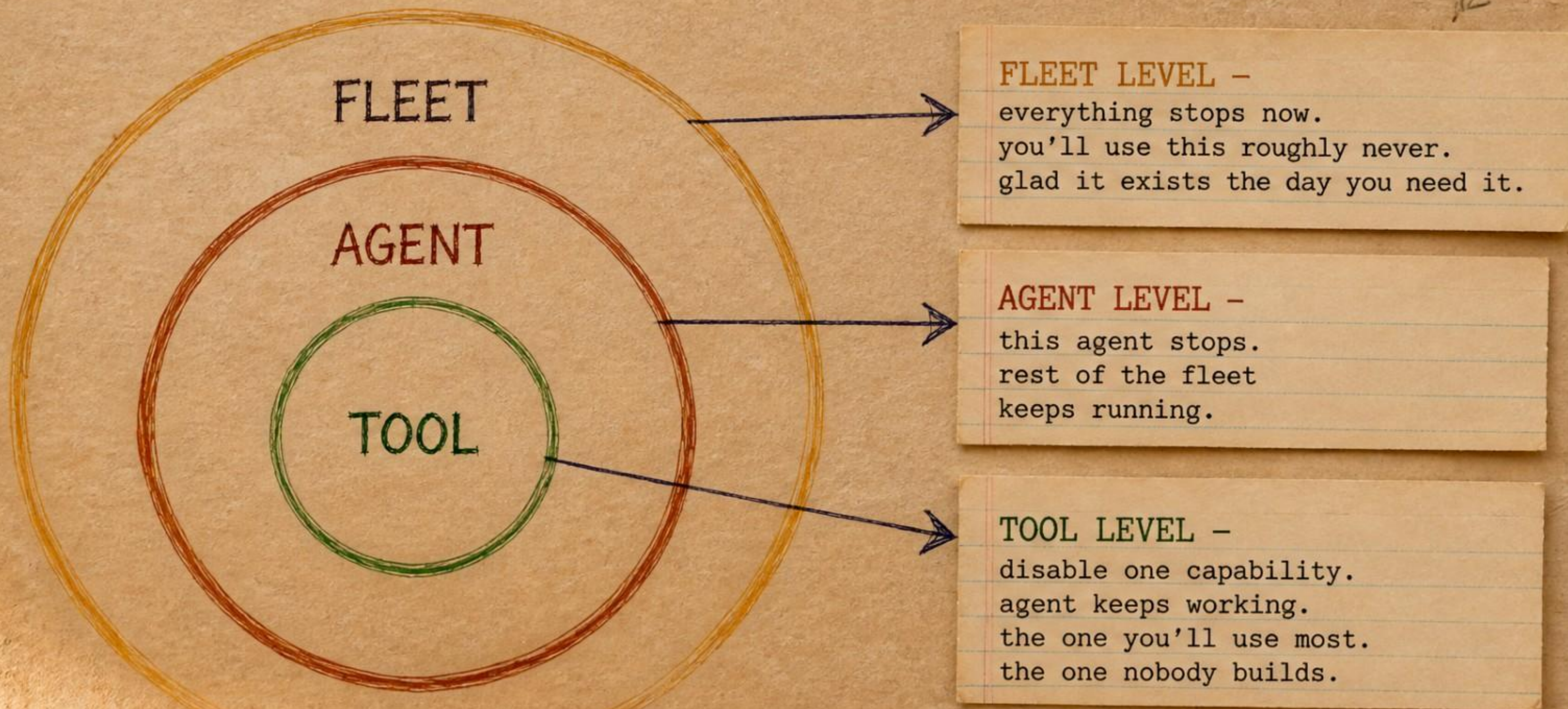
*Approval fatigue: 40 approvals a day by Wednesday,
click-approving batches by Friday. All the friction. None of the safety.*

IN THIS DEMO: exactly ONE tool is gated — the one that texts two humans.

Three Levels of Stop

you climb down when the wind switches

the one
nobody
asks for in
the demo



In my demo — all of this is a flag file and an if-statement.

escalation = softer version. hand the turn to a human, keep the record.


```
$ scout run --agent scout-1
```

```
> intent: notify_hunting_party
> tool.text_writer called with:
  recipients: ["Justin", "Kurt"]
  message: "Sitting Bean Field Saturday, 06:00."
```

[HELD BY POLICY]

Tool: text_writer
Reason: gated_action (reaches_human=true)
Status: PENDING APPROVAL
Queue position: 1 of 1
Awaiting: human decision (approve / deny)

ERRORS: 0

| HELD BY POLICY: 1

 these are DIFFERENT columns on purpose.

```
=====
$ scout kill --agent scout-1 --reason "eval gate red"
```

```
> agent scout-1 receives SIGSTOP
> flag file written: /var/run/scout/killed
> all pending turns cancelled
> fleet status:
  scout-1      STOPPED (killed 15:03:22)
  scout-2      RUNNING
  scout-3      RUNNING
  marine-1     RUNNING
> agent-level stop complete. rest of fleet unaffected.
```



Count the Agents You've Retired

SHIPPED	STILL RUNNING	RETIRED
		?
		0

Nobody is being
measured on how
many they've retired.

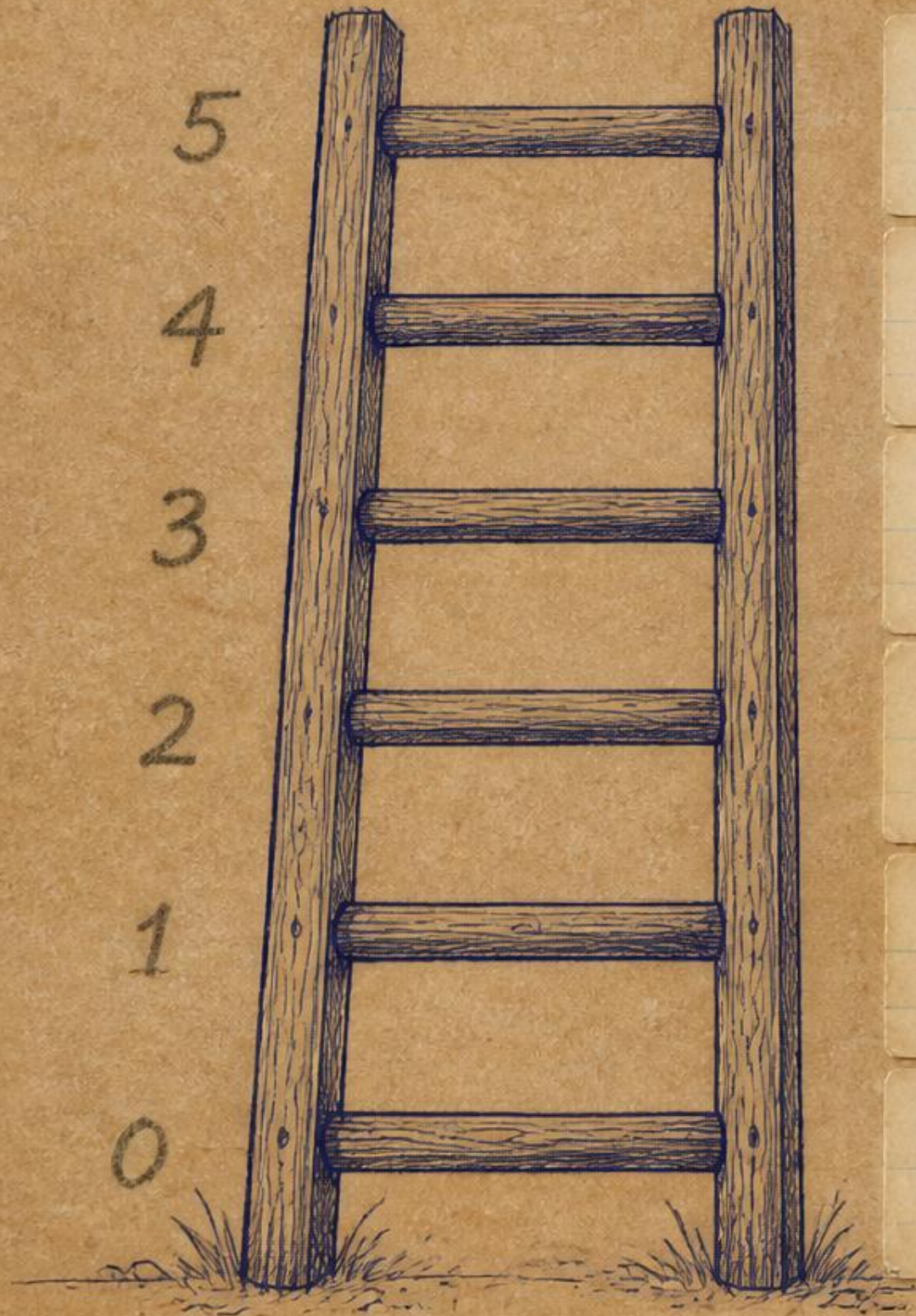
agents that nobody uses / nobody owns /
still holding credentials / still
calling APIs → unmonitored surface area.

Decommissioning is the most mature form of containment.

Five Rungs

locate yourself. honestly. for four seconds.

*if you have agents in prod
with no way to stop them,
you start at rung zero.*



DATA-LAYER INVARIANTS CATCH DRIFT UPSTREAM

Detect the problem before the agent consumes it.

FAILURES TRIGGER CONTROLS AUTOMATICALLY

Hold writes, escalate, degrade capabilities, or stop the agent based on policy.

EVALS RUN AUTOMATICALLY

On system changes and on an appropriate production cadence.

TRACES LAND SOMEWHERE QUERYABLE

You can reconstruct what happened.

TWENTY QUESTIONS WRITTEN DOWN

Golden cases with expected outcomes.



STOP IT

A tested kill mechanism exists.

Built order = capture → compare → contain. Ladder order starts with contain. Deliberate.

Before You Change Anything Next Week

Write down twenty questions. Write down the answers.

→ that's rung one. an afternoon.

*not a program.
not a committee.
one afternoon.*



github.com/bhaydin/scout -
code, cases, invariants



linkedin.com/in/brianhaydin -
tell me what broke

It all runs on a laptop. Steal any of it. I'd rather hear about your failure mode than my own.

Raja Mukerji closes
at 3:45 — Diercks Hall

S25 · payoff · 38:00–39:00

Machines can answer.
Deciding what counts as a **right** answer
— that's ours.

Production AI doesn't fail loudly. It fails plausibly. Nobody files the ticket.

Twenty questions. Write them down. Be the one who files the ticket.

only if live suite fails

The Red Build — Static Reference

```
FACTUAL 8/8 · TEMPORAL 0/5 ·  
RULE 3/3 · REFUSAL 2/2 ·  
WRITE-GATE 2/2
```

GATE: BLOCKED

Suite result.

```
2023: 92% ✓ | 2024: 91% ✓ |  
2025: 93% ✓ | 2026: 33% ✗ |
```

Data-layer invariant.

```
turn.request  
└─ planner  
    └─ tool.retrieve_detections  
        └─ camera.firmware: [2.1, 2.2, 2.2]
```

ERRORS: 0

Trace with smoking gun.

```
[HELD BY POLICY] text_writer  
└─ PENDING APPROVAL
```

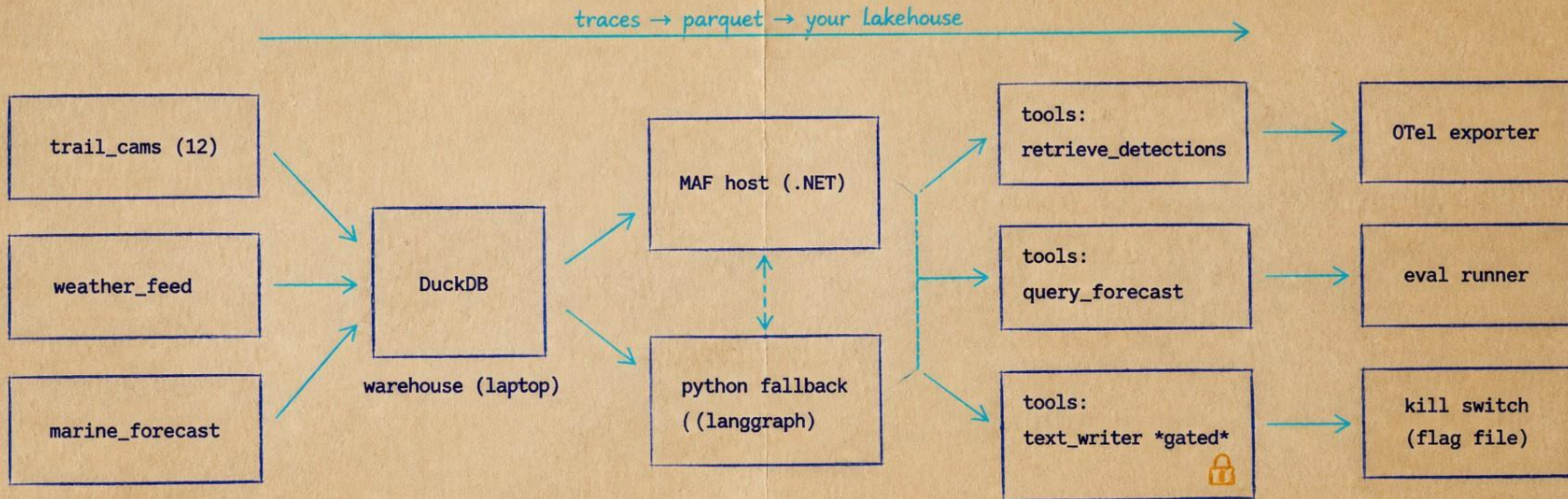
ERRORS: 0 | HELD: 1

Approval gate holding.

recorded for reliability · narrate identical to live demo

Scout — Reference Architecture

do NOT show
from stage →
reads as
vendor pitch



cyan arrows = OpenTelemetry spans • amber = gated action • phosphor-green = passing invariant check

Works on Databricks, Snowflake, DuckDB. Framework doesn't matter — instrumentation does.